Name of the Student	MADDIRALA BALAMURALI KRISHNA
Reg. No	192325015
GitHub Link	https://github.com/krishnalsk/MLA0403-DEEP-LEARNING.git

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

COURSE NAME: Deep Learning **COURSE CODE:** MLA04

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

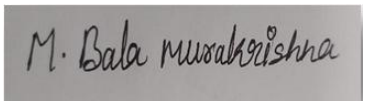
Course Outcome Weightage: 10% Duration: 90 minutes

Assessment Tool Weightage: 30% Mark: 25

Code of Conduct

I *MADDIRALA BALAMURALI KRISHNA(1923250151)*(certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



MADDIRALA BALAMURALI KRISHNA

Signature of the student: Name of the student:

Criteria	Problem Understanding & Req. Analysis (2)	Model Architecture & Design (2.5)	Application of DL Techniques (2.5)	Model Dev. & Implementation Strategy (2)	Performance Analysis & Justification (1)	Total (10)
Weightage	20%	30%	25%	15%	10%	100%
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

QUESTIONS: 05 Total Marks: 25

Q1. Hospital X-ray Transfer Learning (2 marks basis: 2,000 labeled images, normal vs abnormal, pre-trained ResNet)
Design a CNN-based transfer learning model by identifying which layers should be frozen, which layer should be replaced, and how the model should be trained.

Answer 1: CNN-Based Transfer Learning for Hospital X-ray Classification

1.1 Problem Understanding

Only 2,000 labeled chest X-ray images are available for a binary task (normal vs abnormal). Training a deep CNN from scratch on this size dataset will overfit severely. Transfer learning from a network pre-trained on ImageNet (ResNet-50/ResNet-18) lets the model reuse general low-level visual features (edges, textures, blobs) and adapt only the higher-level, task-specific features to radiographic patterns.

1.2 Which Layers to Freeze vs Replace

- Freeze: Stem (conv1, bn1, maxpool) and Layer1–Layer2 of ResNet-50. These encode generic low-level features (edges, gradients, corners, simple textures) that transfer well across domains and are safe to keep frozen with only 2,000 images.
- Fine-tune (unfreeze, low LR): Layer3 and Layer4. These encode mid/high-level, more dataset-specific features; X-ray textures (lung opacities, consolidations) differ enough from ImageNet objects that these blocks benefit from adaptation.
- Replace: The original 1000-way ImageNet FC head is removed entirely and replaced with a new classification head: GlobalAveragePooling2D → Dense(256, ReLU) → Dropout(0.5) → Dense(1, Sigmoid) for binary output.

1.3 Training Strategy (Two-Phase Fine-Tuning)

- Phase 1 (Warm-up): Freeze the entire convolutional base; train only the new head for 5–10 epochs with Adam, LR = 1e-3, so randomly initialized head weights don't destroy pre-trained features.
- Phase 2 (Fine-tune): Unfreeze Layer3–Layer4, drop LR to 1e-5–1e-4 (discriminative learning rates: lower for earlier unfrozen layers), train 15–20 more epochs with early stopping on validation AUC.
- Regularization: Dropout(0.5) in head, weight decay (1e-4), label smoothing, and heavy augmentation (rotation ±10°, horizontal flip, brightness/contrast jitter, slight zoom) — avoid vertical flip since anatomical orientation is meaningful.
- Class imbalance: use class-weighted BCE or focal loss if abnormal/normal ratio is skewed.

1.4 Tools & Evaluation

- Implementation: PyTorch (torchvision.models.resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)) or TensorFlow/Keras (tf.keras.applications.ResNet50).
- Evaluation: Sensitivity/Recall (critical — missing an abnormal case is costly), Specificity, AUC-ROC, confusion matrix; Grad-CAM for clinical explainability.
- Limitation: 2,000 images is still small for medical imaging robustness; k-fold cross-validation and external test-set validation are recommended before deployment.

Q2. DenseNet for Plant Disease Classification
An agricultural organization wants to identify five types of plant diseases from leaf images with limited training samples. Design a DenseNet-based classification model and explain how dense connections help feature reuse.

Answer 2: DenseNet-Based Classification for Plant Disease Detection

2.1 Problem Understanding

Five plant-disease classes must be identified from leaf images with a limited training set. DenseNet is well suited here because its dense connectivity pattern maximizes feature reuse, which is precisely what is needed when labeled data is scarce.

2.2 Model Design

- Backbone: DenseNet-121 pre-trained on ImageNet (4 dense blocks + 3 transition layers).
- Freeze Dense Block 1–2 (generic texture/edge features); fine-tune Dense Block 3–4 and the final transition/BN layer, since leaf-disease lesions require more specific mid/high-level features.
- Replace the classifier head: GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.4) → Dense(5, Softmax).

2.3 Why Dense Connections Help Feature Reuse

- Each layer l receives the concatenation of feature maps from ALL preceding layers in its block (not a summation, as in ResNet), so features computed once are directly accessible to every subsequent layer instead of being re-derived.
- Because each layer only needs to add a small number of new feature maps (growth rate k, typically 12–32), the network stays parameter-efficient — critical for avoiding overfitting on limited data.
- The dense connectivity strengthens gradient flow (implicit deep supervision) and reduces the vanishing-gradient problem, which improves training stability with fewer samples per class.
- Because low-level lesion-texture features from early layers remain directly reachable at the classification layer, the network can distinguish visually similar disease symptoms without re-learning basic patterns for every class — effectively acting like a built-in feature ensemble.

2.4 Training Strategy & Tools

- Heavy augmentation given limited samples: random crop, color jitter (hue/saturation, since disease spots are color-dependent), CutMix/MixUp to synthetically expand the effective dataset.

- Loss: categorical cross-entropy with class weighting for any class imbalance; optimizer Adam, LR = 1e-4 with cosine decay.
- Tools: tf.keras.applications.DenseNet121 or torchvision.models.densenet121(pretrained=True); PlantVillage-style datasets are the common real-world benchmark for this exact use case.
- Evaluation: macro-F1 (equal importance across 5 disease classes), per-class confusion matrix to spot visually confusable diseases.

Q3. LSTM for Student Performance Prediction

Design an LSTM-based model that processes weekly attendance, assignment marks, internal assessment marks, and previous examination scores to predict a student's performance category.

Answer 3: LSTM-Based Model for Student Performance Prediction

3.1 Problem Understanding

Weekly attendance, assignment marks, internal assessment marks, and prior exam scores form a multivariate time series per student. The task is sequence classification: predict a performance category (e.g., Excellent / Good / Average / At-Risk) from the trend across weeks, not just the latest snapshot.

3.2 Data Preparation

- Structure input as a 3D tensor: (batch, timesteps=weeks, features=[attendance%, assignment_marks, internal_marks, prior_score]).
- Normalize each feature (Min-Max or Z-score) since attendance % and marks are on different scales.
- Use a Masking layer or padding for students with variable numbers of recorded weeks so the LSTM ignores padded timesteps.

3.3 Architecture

- Input(timesteps, 4) → Masking → LSTM(64, return_sequences=True, dropout=0.2, recurrent_dropout=0.2) → LSTM(32) → Dense(16, ReLU) → Dropout(0.3) → Dense(num_classes, Softmax).
- Why LSTM: its input/forget/output gates regulate what information is retained or discarded across weeks, letting the model capture longer-term trends (e.g., a gradual attendance decline) while avoiding the vanishing-gradient problem that limits vanilla RNNs.

3.4 Training Strategy & Tools

- Loss: categorical cross-entropy (or ordinal loss, since performance categories are inherently ordered — a practical refinement over plain softmax).
- Optimizer: Adam (LR 1e-3 with ReduceLROnPlateau), EarlyStopping on validation loss, batch size 32.
- Tools: TensorFlow/Keras (LSTM, Masking) or PyTorch (nn.LSTM with pack_padded_sequence for variable-length sequences).
- Evaluation: weighted F1, confusion matrix; SHAP/attention weighting on timesteps can highlight which week most influenced the prediction — useful for early-intervention alerts.
- Limitation: LSTMs need a reasonable number of historical weeks per student; for very short sequences (e.g., only 2–3 weeks in), a simpler model (e.g., gradient-boosted trees on aggregated features) may outperform it — worth A/B testing.

Q4. Bidirectional RNN for Sentiment Classification

Design a Bidirectional RNN model to classify customer reviews as positive, negative, or neutral, and identify the major processing stages from input text to final prediction.

Answer 4: Bidirectional RNN for Customer Review Sentiment Classification

4.1 Problem Understanding

Reviews must be classified as positive, negative, or neutral. Sentiment-bearing words are often context-dependent in both directions — e.g., in "not very good", the word "good" is negated by a word that appears BEFORE it, while in "good, but not as good as last time" the qualifier appears AFTER. A unidirectional RNN only sees past context; a Bidirectional RNN processes the sequence both left-to-right and right-to-left, giving each word representation access to full sentence context.

4.2 Processing Pipeline (Input → Prediction)

- 1. Text preprocessing: lowercase, remove punctuation/HTML, optional stopword handling (keep negation words like "not"), tokenization.
- 2. Sequence encoding: Tokenizer → integer sequences → padding/truncation to fixed length.
- 3. Embedding layer: maps tokens to dense vectors — use pre-trained GloVe (300d) or FastText embeddings for better generalization with limited labeled reviews, or a learned embedding layer if the dataset is large.
- 4. Bidirectional recurrent layer(s): two LSTM/GRU passes (forward and backward) whose hidden states are concatenated at each timestep.
- 5. Pooling/aggregation: concatenate final forward and backward hidden states, or apply global max/average pooling over all timestep outputs.
- 6. Dense classification head: Dense(32, ReLU) → Dropout(0.5) → Dense(3, Softmax) → argmax for final label.

4.3 Architecture Summary

- Embedding(vocab_size, 300, pretrained_weights=glove) → Bidirectional(LSTM(128, return_sequences=True)) → Bidirectional(LSTM(64)) → Dense(32, ReLU) → Dropout(0.5) → Dense(3, Softmax).

4.4 Training Strategy & Tools

- Loss: categorical cross-entropy with class weights, since the neutral class is typically under-represented in real review datasets.
- Optimizer: Adam, LR 1e-3, gradient clipping (norm=5) to stabilize RNN training.
- Tools: TensorFlow/Keras Bidirectional wrapper, Keras Tokenizer + pad_sequences, pretrained GloVe/FastText vectors; a modern production alternative is fine-tuning a pretrained transformer (e.g., DistilBERT) for higher accuracy at higher compute cost.
- Evaluation: macro-F1 (fairer than accuracy across imbalanced 3-class sentiment), confusion matrix to check positive/negative vs neutral confusion.

Q5. Encoder–Decoder for English–Hindi Translation

Design an Encoder–Decoder model and describe the role of the encoder, decoder, hidden representation, and output layer in the translation process.

Answer 5: Encoder–Decoder Model for English–Hindi Translation

5.1 Problem Understanding

Machine translation is a sequence-to-sequence problem where input and output lengths differ and the full input meaning must be captured before generating output — exactly the scenario the Encoder–Decoder architecture was designed for.

5.2 Architecture and Role of Each Component

- Encoder: A (Bidirectional) LSTM/GRU reads the English sentence token-by-token (after subword tokenization, e.g., Byte-Pair Encoding/SentencePiece, plus word embeddings) and produces a sequence of hidden states; its final hidden and cell states summarize the entire input sentence.
- Hidden representation (context vector): The compressed representation of the source sentence passed from encoder to decoder. In a basic Seq2Seq this is just the encoder's final state; in practice this fixed-length bottleneck is replaced with an Attention mechanism (Bahdanau/Luong) that lets the decoder access a weighted combination of ALL encoder hidden states at every decoding step — essential for longer sentences where a single fixed vector loses information.
- Decoder: Another LSTM/GRU, initialized with the encoder's final states, generates the Hindi sentence one token at a time. At each step it conditions on the previously generated token (teacher forcing with the ground-truth token during training; its own previous prediction during inference) and the (attention-weighted) context.
- Output layer: A Dense layer + Softmax over the Hindi vocabulary at each decoder timestep produces a probability distribution; the next token is chosen via greedy argmax or, for better quality, beam search.

5.3 Training Strategy & Tools

- Tokenization: subword units (BPE/SentencePiece) are essential for Hindi given its rich morphology and to handle out-of-vocabulary words on both sides.
- Loss: token-level categorical cross-entropy with padding masked out; teacher forcing ratio can be annealed during training.
- Tools: TensorFlow/Keras or PyTorch Seq2Seq with Bahdanau/Luong attention; OpenNMT or Hugging Face `transformers`/`MarianMT` for a production-grade pipeline; in current industry practice, an actual deployment for English–Hindi would typically use a pretrained Transformer model (e.g., IndicTrans2, mBART-50, NLLB-200) fine-tuned on parallel corpora (e.g., Samanantar, IIT Bombay EN-HI corpus) rather than training a vanilla LSTM Seq2Seq from scratch.
- Evaluation: BLEU / chrF score against reference translations; human evaluation for fluency and adequacy, since automatic metrics alone are known to be imperfect proxies for translation quality.

5.4 Limitations & Future Scope

- Plain LSTM Seq2Seq struggles with long sentences and rare words even with attention; Transformer-based architectures (self-attention, parallelizable training) are now the practical industry standard and should be the recommended upgrade path.
- Low-resource issue for Hindi parallel data can be mitigated with back-translation and multilingual pretrained models.